These are the key and the mapping function that the data obfuscator and data enricher need to route the messages. With them, they can create a group router, as shown in the following snippet. With this strategy, two messages with the same ID have the same hash and, thus, the same routee. Note that to spawn the router, the actor uses `.spawnAnonymous`:

```
object DataEnricher {
  ...

  def apply() = Behaviors.setup[Command] { context =>

    val router = context.spawnAnonymous {
      Routers
        .group(Aggregator.serviceKey)                              ❶
        .withConsistentHashingRouting(10, Aggregator.mapping)      ❷
    }
  ...
}
```

❶ Points to the service key to find registered aggregators

❷ Uses factor 10 to spread the load more evenly on the aggregators. Uses a mapping function to allow the creation of a hash.

It is a good practice to name actors for debugging purposes, but this is just a small example chosen here for simplicity. If you look at the Akka source code, many specs use this kind of spawning for this reason.

NOTE The specific logic of how the aggregate merges the messages has little to do with routing. If you are interested, you can find the code in the RoutingExamplesSpec.scala file, which also contains tests.

The data enricher and the obfuscator have very similar logic. Here is the rest of the enricher, which gives a complete view of how the actor uses the router: